Uni-Renderer: Unifying Rendering and Inverse Rendering Via Dual Stream Diffusion

Zhifei Chen[1,*], Tianshuo Xu[1,*], Wenhang Ge[1,*], Leyi Wu[1], Dongyu Yan[1], Jing He[1], Luozhou Wang[1], Lu Zeng[3], Shunsi Zhang[3], Yingcong Chen[1,2,†],
[1]HKUST(GZ), [2]HKUST, [3]Quwan
{zchen379, txu647}@connect.hkust-gz.edu.cn; yingcongchen@ust.hk

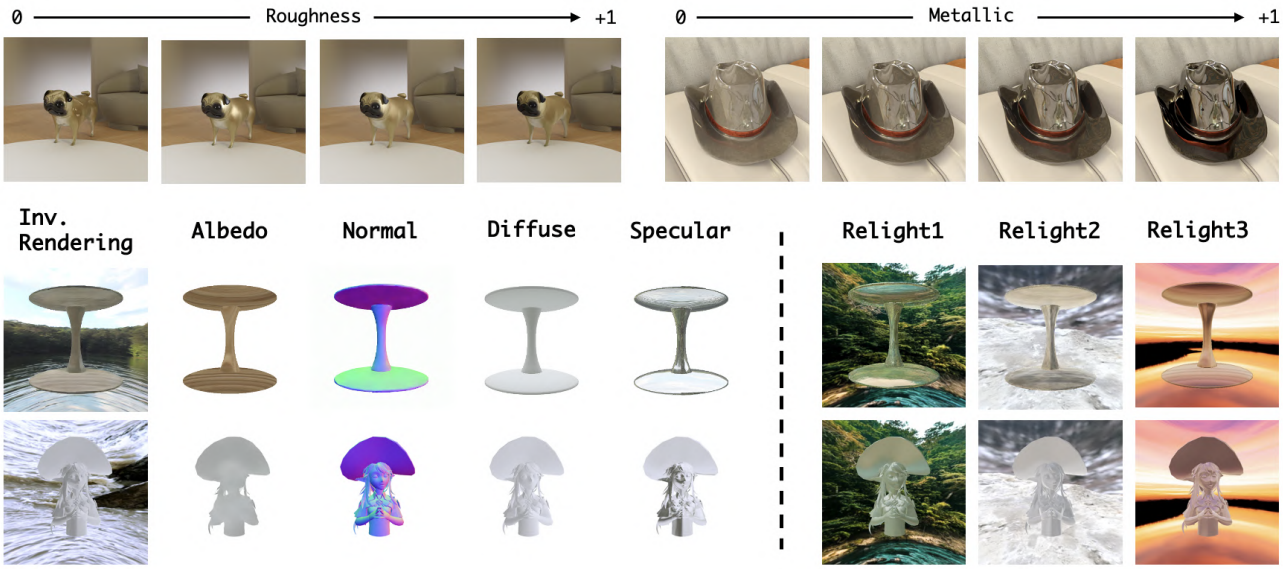

Figure 1. Our framework, Uni-renderer, empowers the generative model to function both as a renderer and an inverse renderer by approximating the rendering equation using a data-driven approach. Given intrinsic attributes, Uni-renderer generates photo-realistic images, functioning as a renderer. When provided with a single RGB image, it effectively decomposes the intrinsic properties, functioning as an inverse renderer. **Top:** Uni-renderer generates smooth variations as renderer. Setting the roughness value to 1.0 results in the "dog" case, shown at the top, lacking specular highlights. Conversely, setting the metallic value to 1 makes the "hat" case appear metallic. **Bottom Left:** When functioning as an inverse renderer, Uni-renderer decomposes the intrinsic properties of a single RGB image. **Bottom Right:** Uni-renderer generates relighting results under different environment lighitngs.

Abstract

Rendering and inverse rendering are pivotal tasks in both computer vision and graphics. The rendering equation is the core of the two tasks, as an ideal conditional distribution transfer function from intrinsic properties to RGB images. Despite achieving promising results of existing rendering methods, they merely approximate the ideal estimation for a specific scene and come with a high computational cost. Additionally, the inverse conditional distribution transfer is intractable due to the inherent ambiguity. To address these challenges, we propose a data-driven method that jointly models rendering and inverse rendering as two conditional generation tasks within a single diffusion framework. Inspired by UniDiffuser, we utilize two distinct time schedules to model both tasks, and with a tailored dual streaming module, we achieve cross-conditioning of two pre-trained diffusion models. This unified approach, named Uni-Renderer, allows the two processes to facilitate each other through a cycle-consistent constrain, mitigating ambiguity by enforcing consistency between intrinsic properties and rendered images. Combined with a meticulously prepared dataset, our method effectively decomposition of intrinsic properties and demonstrating a strong capability to recognize changes during rendering.

∗ Equal contribution. †Corresponding Author.
This project is supported by Quwan Technology.

1. Introduction

Physically based rendering is crucial in computer vision and graphics for producing photorealistic 2D images from 3D models, textures, and lighting setups. This process is essential in animation [43], and architectural visualization [23]. At its core, the rendering equation [18] models the flow of light energy within a scene, offering an ideal conditional distribution transfer function for rendering. Monte Carlo light-transport simulations [34], utilizing path tracing [24], are commonly employed to evaluate the rendering equation. However, its recursive tracing incurs significant computational demands, posing challenges for real-time implementation. Inverse rendering, which aims to deduce geometric, material, and lighting information from images, has long been studied due to its under-constrained nature [10]. This approach allows the reconstructed 3D model to be directly integrated into rendering engines, thereby playing a critical role in downstream applications such as game production [25].

Recent advancements employ differentiable rendering for 3D representations [5, 45] or focus on direct decomposition of intrinsics using 2D object priors [7, 11]. These approaches use differentiable renderer to integrate intrinsic properties into RGB images, facilitating loss computation against ground-truth images for optimization. For instance, MaterialGAN [11] leverages StyleGAN2 [19] to synthesize realistic material properties using a large-scale, spatially-varying material dataset [7], while other works [4, 22, 44, 46] utilize diffusion models to achieve better results. Nevertheless, the ambiguity between geometry, materials, and lighting still hinders the effectiveness of decomposition. The mapping from observed images back to intrinsic properties is not one-to-one, leading to ambiguity and suboptimal performance. The most similar work to ours is RGB2X [47], which performs both forward and inverse rendering using two separate diffusion models. However, it fails to establish a connection between the two processes, treating them independently. Ambiguity issue still exists.

To alleviate this issue, we propose to jointly learn the two distributions together. By integrating rendering and inverse rendering into a single framework from a multi-task learning perspective, the two processes can facilitate each other. This joint learning allows us to use the inverted intrinsic properties to perform another cycle of rendering, effectively creating a cycle-consistent constrains. This cycle rendering can mitigate the ambiguity problem by enforcing consistency between the intrinsic properties and the rendered images, leading to improved performance.

In this work, we explicitly model the rendering process and inverse rendering as two conditional generation tasks. Inspired by UniDiffuser [1], we utilize two distinct time schedules to model both conditional generation tasks within a single diffusion pipeline. By cross-conditioning two pre-trained diffusion models through a dual streaming module, we achieve both rendering and inverse rendering in a unified framework.

To summarize, we introduce Uni-renderer which utilizes a pre-trained diffusion model to handle both rendering and inverse rendering in a unified framework. The key contributions of our method are as follows:

- We propose a data-driven method to approximate the rendering equation. By modeling rendering and inverse rendering as two conditional generation tasks, we design a unified diffusion model with a tailored dual stream module for cross-condition generation.
- We render a synthetic dataset of different fine-grained material edits using 200K 3D objects with randomized intrinsic attributes by varying one of the rendering elements at a time while keeping the others constant.
- Extensive experiments validate the effectiveness of our method, achieving robust disentangling of intrinsic properties and demonstrating a strong capability to recognize changes in rendering.

2. Related Work

2.1. Rendering

Rendering, the process of generating 2D images from 3D models, synthesizes raw data from a 3D scene—including geometry, materials, and lighting—using the rendering equation [18]. This task is typically performed by rendering engines, such as Blender [14] and Unity [12], which serve as the technological intermediaries converting 3D models into 2D images or videos. However, achieving photo-realistic 2D images often involves recursive ray tracing [38] or path tracing [24], techniques that closely approximate the rendering equation but are time-consuming. Although several GPU-based acceleration methods have been proposed [8, 13, 21], and existing engines have incorporated these tools, real-time rendering continues to pose significant challenges. In contrast to traditional rendering approaches, our method considers the diffusion model as an alternative renderer, which utilizes geometry, materials, and lighting as conditional cues to produce photo-realistic 2D images without the need for recursive tracing.

2.2. Inverse Rendering

Inverse rendering [2, 33], the task of decomposing an image's appearance into intrinsic properties such as geometry, material, and lighting, remains a longstanding and severely ill-posed problem in computer vision and graphics. Some approaches employ differentiable renderers on 3D representations [5, 20] to directly optimize these properties using image losses. Advancements in volume rendering, as utilized in Neural Radiance Fields (NeRFs) [29] for 3D reconstruction from multi-view 2D images, have led most

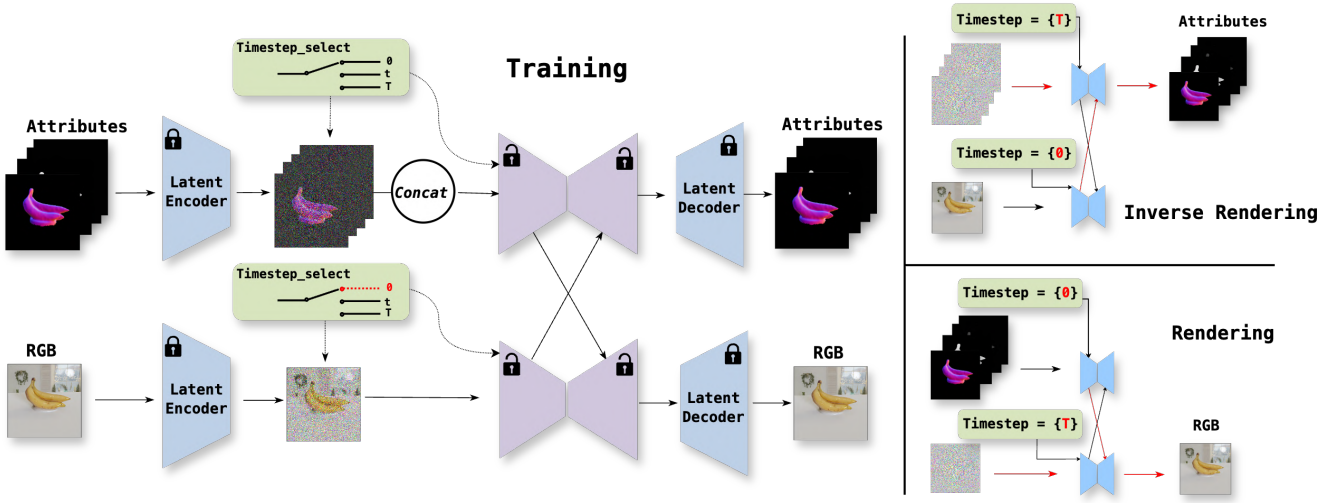

Figure 2. **The overview of our pipeline.** During training, both attribute and RGB images are input to a unified model with pre-trained VAE encoders. The timestep selector plays a crucial role by adjusting the timesteps for each branch. Specifically, it ensures that one branch (either the attribute or RGB) has a timestep of 0, while the other branch selects a timestep from $t \in [0, T]$. This mechanism allows our model to effectively learn the conditional distributions $q(\mathbf{x_0}|\mathbf{y_0})$, $q(\mathbf{y_0}|\mathbf{x_0})$ in alternating iterations. During rendering and inverse rendering, the corresponding conditions are input to the model with a timestep of 0, and the attributes/RGB images are generated through a sampled noise. (The VAE encoder and decoder are omitted for simplicity.)

methods [9, 26, 27, 45, 48] to leverage multiple images for reconstruction and subsequent estimation of materials and lighting. Additionally, they require per-object optimization and struggle to generalize across different objects, limiting their efficiency in fast estimation. MaterialGAN [11] utilizes a StyleGAN2-based model for generating intrinsic properties and a renderer for novel view synthesis under specified lighting conditions. Similarly, SIC [7] employs an encoder-decoder network for single-image-based inverse rendering, where rendering loss is used for optimization. Nonetheless, these efforts predominantly focus on planar surfaces and still necessitate a renderer to integrate predicted attributes into the final rendered image. [4, 22, 44, 46, 47] utilize diffusion models to achieve better results. However, these methods have not achieved optimal performance due to the inherent ambiguity involved in transferring distributions from images to intrinsic properties. Our method effectively mitigates the ambiguity among intrinsic properties by introducing a dual-stream diffusion pipeline and combined with a meticulously prepared dataset.

2.3. Diffusion Models

Diffusion Models [15, 32, 35, 39, 40] belong to the class of probabilistic generative models that progressively destruct data by injecting noise, and then subsequently learn to reverse this process in order to generate new samples. Alchemist [37] leverages a pre-trained diffusion model to achieve material control over a given object. [42] adopts a collaborative diffusion pipeline to decompose physically-based rendering (PBR) material properties from RGB. However, these works focus either on rendering or inverse rendering and fail to yield a unified model to fit in real-world usages. Alternatively, we design a dual stream diffusion model as both a renderer and an inverse renderer. During the inverse rendering stage, it takes a single RGB image as input and disentangles all the intrinsic properties. For the rendering stage, all of the intrinsic properties including material, geometry, and lighting conditions will be taken and rendered via the same model.

3. Approach

Given a single RGB image $\mathbf{I}$, inverse rendering aims at decomposing it into intrinsic attributes including metallic m, roughness r, albedo $\mathbf{a}$, surface normal $\mathbf{n}$, specular lighting $\mathbf{s}$ and diffuse lighting $\mathbf{d}$, which is formulated as $\{m, r, \mathbf{a}, \mathbf{n}, \mathbf{s}, \mathbf{d}\} = \text{InverseRenderer}(\mathbf{I})$. For later reference, we denote the combination of attributes as $\mathbf{C}$. Renderer takes attributes as input and renders them into RGB image denoted as $\mathbf{I} = \text{Renderer}(\mathbf{C})$. We formulate a dual stream diffusion model as renderer and inverse renderer simultaneously. We start by briefly revisiting physically based rendering (PBR) and Unidiffuser [1] in Section 3.1. Next, we introduce our dual stream diffusion framework in Section 3.2. We elaborate on the data preparation in Section 3.3.

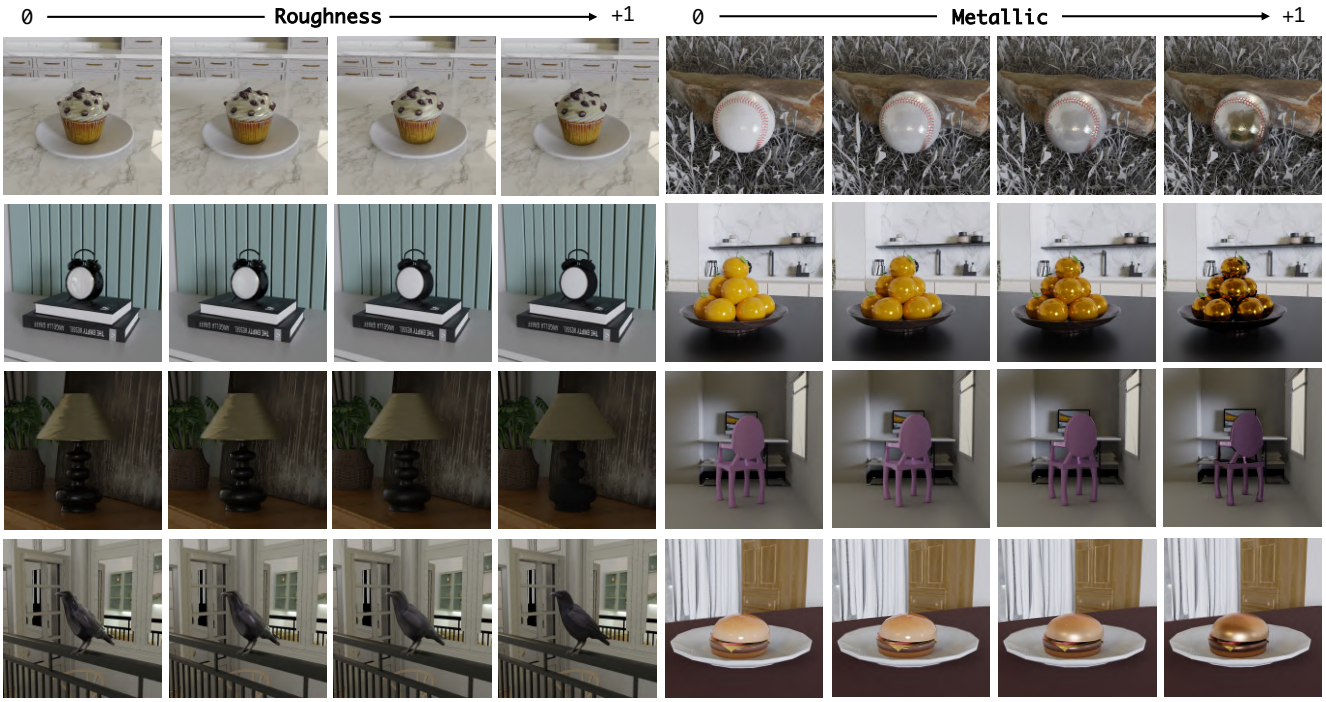

Figure 3. We demonstrate smooth changes via rendering for different metallic and roughness strengths. The rendering is performed giving different combinations of the attributes. When the roughness value was set to 1, the cake and clock case shown in the top left are without specular highlights. When the metallic value was set to 1, the orange and baseball cases appeared to be metallic and revealing object illumination. **Best viewed in color.**

3.1. Preliminaries

Physically Based Rendering. Physically-based rendering is a computer graphics approach that seeks to render images by modeling the interaction between lights and materials in the real world. At its core, the rendering equation [18] describes the light energy flow in the scene, formulated as

$$L(\mathbf{x}, \boldsymbol{\omega_o}) = \int_{\Omega} f(\mathbf{x}, \boldsymbol{\omega_o}, \boldsymbol{\omega_i}) L_i(\mathbf{x}, \boldsymbol{\omega_i})(\boldsymbol{\omega_i}, \mathbf{n}) d\boldsymbol{\omega_i}, \quad (1)$$

where $\boldsymbol{\omega_o}$ is the viewing direction of the outgoing light, f is the BRDF properties, L_i is the incident light of direction $\boldsymbol{\omega_i}$ sampled from the upper hemisphere Ω, and $\mathbf{n}$ is the surface normal. The rendering equation is the key to handle both rendering and inverse rendering problems. Given intrinsic properties, the rendering equation generates photo-realistic images. Given an image, a rendering equation can also be used to optimize intrinsic properties.

Unidiffuser. UniDiffuser [1], a unified diffusion model that models all distributions at the same time, injects noise ϵ^x and ϵ^y to a set of paired image-text data $(\mathbf{x_0}, \mathbf{y_0})$ and generates noisy data $\mathbf{x_t^x}$ and $\mathbf{y_t^y}$, where $0 \leq t_x, t_y \leq T$ represent two individual timesteps. It then trains a joint noise prediction network $\epsilon_\theta(\mathbf{x_{t^x}}, \mathbf{y_{t^y}}, t_x, t_y)$ to predict the noise ϵ^x and ϵ^y by minimizing the mean squared error loss:

$$\mathop{\mathbb{E}}_{\epsilon^{\mathbf{x}}, \epsilon^{\mathbf{y}}, \mathbf{x_0}, \mathbf{y_0}} \left[\| [\epsilon^{\mathbf{x}}, \epsilon^{\mathbf{y}}] - \epsilon_\theta(\mathbf{x_{t^x}}, \mathbf{y_{t^y}}, t_x, t_y) \|^2 \right], \quad (2)$$

By predicting $\epsilon_\theta(\mathbf{x_{t^x}}, \mathbf{y_{t^y}}, t_x, t_y)$ for any t_x and t_y, UniDiffuser learns all distributions related to $(\mathbf{x_0}, \mathbf{y_0})$. This includes all conditional distributions: $q(\mathbf{x_0}|\mathbf{y_0})$, $q(\mathbf{y_0}|\mathbf{x_0})$, and those conditioned on noisy input, for example $q(\mathbf{x_0}|\mathbf{y_{t^y}})$ and $q(\mathbf{y_0}|x_{t^x})$, for $0 \leq t_x, t_y \leq T$. Learning a conditional distribution can be seen as learning a distinct task. From a multitask learning perspective, due to limited network bandwidth, learning many tasks simultaneously (i.e., fitting all distributions to a single network) may result in task competition or task conflict, ultimately leading to sub-optimal performance. For rendering and inverse rendering, we exclusively modeled the two conditional distributions $q(\mathbf{x_0}|\mathbf{y_0})$, $q(\mathbf{y_0}|\mathbf{x_0})$ to resolve the aforementioned issue and enhance the performance.

3.2. Uni-Renderer

Our design incorporates a pre-trained RGB image model and a PBR model, the two were tightly linked to one another via a dual streaming technique aligning the RGB model's information with the PBR model's information. The overall framework is shown in Figure 2. Next, we will discuss each sub-module in the pipeline in detail.

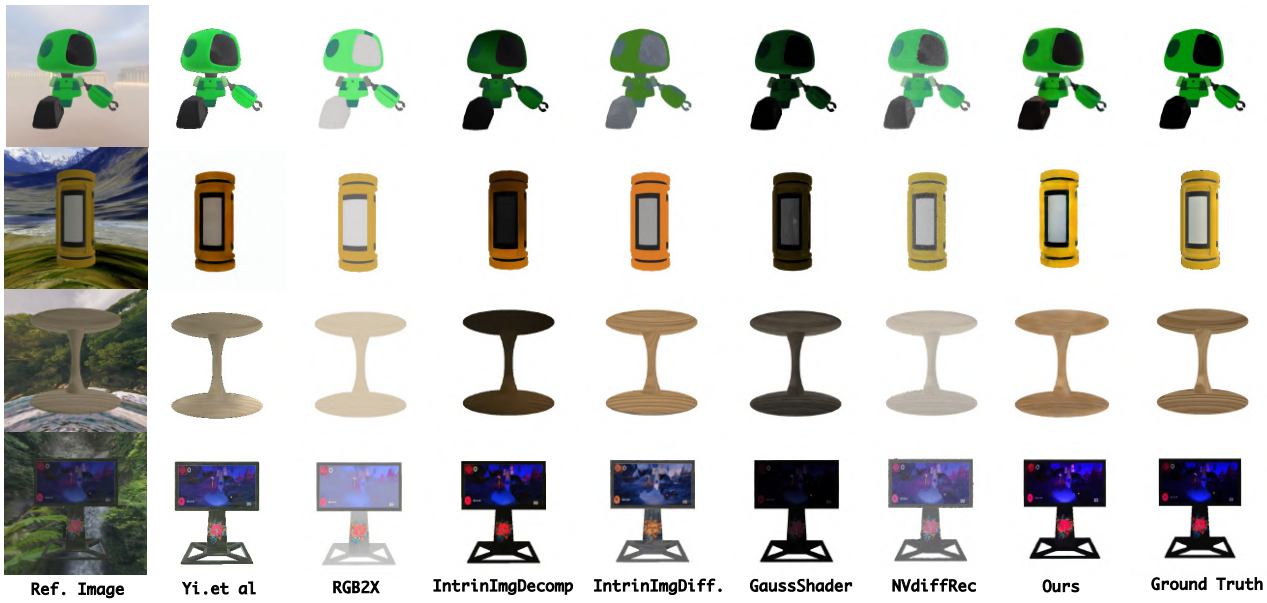

Figure 4. **Albedo comparison.** Albedo Comparison of Uni-Renderer with baseline methods. We compared 4 learning-based methods and 2 optimization-based methods. Among all, Uni-renderer yields the most realistic results. **Best viewed in color.**

Latent Preparation. Recent advancements have benefited greatly from the pre-trained VAE [36], which downsample an RGB image into lower dimensional latent. We used separate VAE for encoding $[m, r]$, albedo $\mathbf{a}$, surface normal $\mathbf{n}$ and environment lighting $\mathbf{s}$, $\mathbf{d}$ before we channel-wisely concatenate them and feed them into the model. It is worth noting that attributes like metallic and roughness, often come with a scalar value, and cannot be directly encoded. Thus, we generate additional masks to binarize the two values into gray-scale images. Instead of having two separate VAE for m and r, We formed channel triplets a_{material}, consisting of $[m, r, \mathbf{m}]$ and process those with the RGB VAE, where $\mathbf{m}$ is the mask This design allows us to reduce the bandwidth overhead within the diffusion model and further improve generation quality.

Conditional Distributions Modeling. To achieve rendering and inverse rendering within a single model, we borrowed ideas from [1], and adopted two different timesteps $t_{\text{attributes}}$, t_{RGB} from three timesteps choices $\{0, t, T\}$ to train a joint noise prediction network. It is worth noting that since our unified model only requires modeling two conditional distributions, we ruled out the choices where both $t_{\text{attributes}}$ and t_{RGB} take on t or T and make sure there is always a timestep equal to 0. Formally,

$$(t_{\text{attributes}},\ t_{\text{RGB}}) = \begin{cases} (0,\ \tilde{t}) & \text{during rendering} \\ (\tilde{t},\ 0) & \text{inverse} \end{cases}, \quad (3)$$

where

$$\tilde{t} = \begin{cases} t & \text{with probability } p \\ T & \text{otherwise} \end{cases}. \quad (4)$$

The pseudo-code for generating timesteps can be found in supplementary material.

Dual Stream Diffusion. We designed a dual diffusion network to facilitate the unified model and adopt the $\mathbf{x}_0$-prediction in the diffusion process. **Training**: During training, as shown in Figure 2, the upper branch of the model dealt with the intrinsic attributes only, leaving only RGB for the lower branch. The timestep selection module will determine if a rendering/inverse rendering iteration has occurred at a probability p. During a rendering iteration, the clean attributes $\mathbf{C}$ will be used as conditions to denoise the noisy RGB image $\mathbf{I}$, the same will be applied to the inverse rendering iteration except that the noisy attributes are being fed to the model. **Inference**: At the inference stage, we provided attributes as conditions to the model and gradually denoise the RGB, and vice versa. The effectiveness of the dual stream framework is demonstrated in Sec 4.3.

Cycle-Consistent Constrain. To alleviate the inherent ambiguity problem, we introduce a cycle-consistent constraint within our unified framework, where its incorporation is straightforward with our pipeline. During training, we use the model's predicted inverse results to perform an additional cycle of rendering. The re-rendered outputs are then utilized in the loss calculation to further optimize the inverse

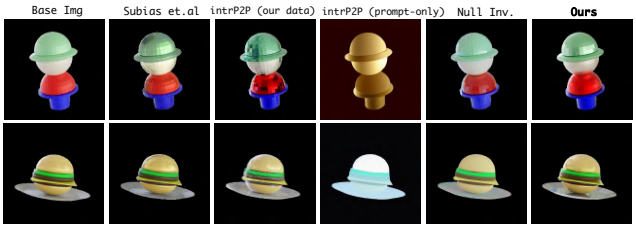

Figure 5. **Qualitative comparison.** Rendering Comparison of Uni-Renderer with baseline methods. The base images are with a metallic value of 0.5. The comparison is made with a higher metallic value of 1.0. **Best viewed in color.**

rendering process. Formally,

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_0,\boldsymbol{\epsilon},t}\left[\left\|\mathbf{x}_0 - \hat{\mathbf{x}}_0\left(\hat{\mathbf{x}}_{\text{rgb}}, t, \mathbf{C}\right)\right\|^2\right], \tag{5}$$

where $\hat{\mathbf{x}}_0\left(\mathbf{x}_t, t, \mathbf{C}\right)$ is the model's prediction of the original rendering output $\mathbf{x}_0$, conditioned on the noisy input $\hat{\mathbf{x}}_{\text{rgb}}$, timestep t, and conditioning attribute information $\mathbf{C}$. The effectiveness of the cycle-consistent constrain is demonstrated in Sec 4.3.

3.3. Data Preparation

We prepare our training data with 3D data from Objaverse [6], consisting of synthetic 3D assets. We sampled 200K assets from Objaverse. For each 3D asset in Objaverse, we rendered the 2D images, and materials map (i.e., metallic, roughness, albedo) by changing the metalness and roughness (range from 0 to 1 with step 0.1). We also randomly select a 2D environment map from a pool of 20K natural scenes from a subset of LHQ-1024 [17] and format them into an RGB style for providing lighting. For each object, we have 121 pairs with different metallic, roughness and lighting. The rendered images are of resolution 1024×1024, and the camera poses are fixed in the front of the object. We also rendered 100 objects that were unseen during training to test the model generalization ability.

4. Experiments

We present qualitative and quantitive analysis in Section 4.1 to demonstrate the rendering capabilities of our model. The inverse rendering results can be found in Section 4.2.

4.1. Rendering

In Figure 3, we demonstrate the effectiveness of our model in generalizing rendering techniques by utilizing different combinations of material attributes. Using our dual streaming pipeline, we start off by passing a reference image to perform inverse rendering. Then update the metallic, roughness value and finally re-render into an RGB image with updated materials.

Table 1. **Quantitative results on rendering.** Metrics for the prompt-only InstructPix2Pix* trained on our data, Subias et.al and our proposed method computing the PSNR, and LPIPS.

Methods	Metallic		Roughness	
	PSNR↑	LPIPS↓	PSNR ↑	LPIPS↓
InPix2Pix* [3]	24.25	0.1032	24.43	0.1056
Subias et.al [41]	28.09	0.0954	28.13	0.0817
Ours	**30.72**	**0.0763**	**31.68**	**0.0695**
Ours w/o unified	27.33	0.0932	29.12	0.0987
Ours w/o re-render	28.72	0.0824	28.93	0.0833

Roughness. As the roughness increases, the output demonstrates the elimination of specular highlights, replaced by an estimate of the base albedo. Conversely, reducing the roughness amplifies the highlights, as evidenced in the cases of the crow and the lamp in Figure 3.

Metallics. The increase of the metallic strength in both the baseball and oranges in Figure 3 reduces the significance of the base albedo and enhances the lighting effects on the surfaces. In contrast, reducing the metallic strength reverses this effect.

Analysis. We compare our model to other material editing baselines: Subias et al. [41], Prompt-to- Prompt with Null-text Inversion (NTI) [30], and InstructPix2Pix [3] in Figure 5. We fine-tuned the InstructPix2Pix prompt-based approach with our dataset. Subias et al.'s method results in exaggerated material changes as their objective is perceptual, not physically-based material rendering. Null-text inversion and InstructPix2Pix trained on our dataset with a prompt-only approach, they either incorrectly modified the background or produces minimum changes. Our method renders the photo-realistic results, introducing the specular highlights while retaining the geometry and illumination effects. Additionally, the quantitative results with the aforementioned baselines are shown in Table 1.

4.2. Inverse Rendering

We use Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index Measure (SSIM), and Learned Perceptual Image Patch Similarity (LPIPS) as metrics to evaluate the image quality of the albedo, relighting images, and novel view synthesis. We use Cosine Similarity for normal evaluation. Roughness and metallic estimation are evaluated using Mean Squared Error (MSE). Detailed configurations of the comparisons are discussed in the supplementary file.

4.2.1. Lighting Estimation

We compare our model with NVdiffRec [31], Gaussian-Shader [16] in terms of specular and diffuse lighting decomposition, the results can be found in Figure 6 and in Table 3. In Figure 6, we demonstrate the relighting ability of our proposed model given different environmental lighting. To achieve this, we updated the specular, diffuse, and environment maps of the given objects. The rendered outputs

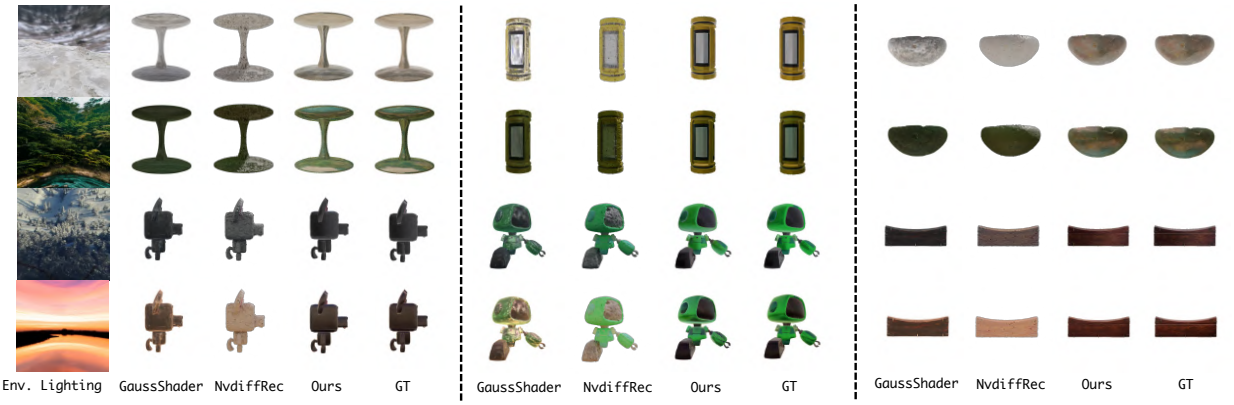

Figure 6. **Relighting Comparsion** The relighting comparison is performed on validation objects. We first inverse render the input RGB to acquire the intrinsics, and then we updated the lighting information to get the relighting results. The leftmost column is the reference environment lighting. **Best viewed in color.**

Table 2. **Quantitative results on inverse rendering.** For quantitative comparison, we have included both data-driven methods and optimization-based baselines. "Ours w/o unified" is trained without the unified framework, and "Ours w/o constrain" is trained without the cycle-consistent constrain term.

Method	Albedo			Metallic	Roughness	Normal
	PSNR↑	SSIM↑	LPIPS↓	MSE↓	MSE↓	cos-simi↑
Yi *et al*. [46]	20.93	0.8960	0.1024	-	-	-
IntrinsicAny. [44]	22.67	**0.9218**	0.0633	-	-	-
Wonder3d [28]	-	-	-	-	-	0.834
Intrin.ImgDecomp. [4]	21.91	0.8934	0.0659	-	-	-
Intrin.ImgDiff. [22]	21.83	0.9060	0.0632	0.1920	0.1315	-
RGB2X. [47]	18.15	0.8829	0.0851	-	-	0.871
NvDiffrec [31]	13.56	0.8695	0.1243	0.3164	0.2956	0.631
GaussianShader [16]	16.55	0.8640	0.0906	0.3421	0.3714	0.908
Ours	**23.20**	0.9182	**0.0532**	**0.1182**	**0.1037**	**0.928**
Ours w/o unified	18.62	0.8846	0.0833	0.1632	0.1391	0.867
Ours w/o constrain	21.20	0.8934	0.0602	0.1391	0.1304	0.922

Table 3. **Quantitative results on relighting.** We have compared the inverse rendering results on lighting estimation in our left-out test set, "Ours w/o unified" is trained without the unified framework, and 'Ours w/o constrain" is trained without the cycle-consistent constrain.

Method	Specular			Diffuse			Relighting		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
NvDiffrec [31]	14.12	0.8822	0.1782	15.20	0.8834	0.1833	21.99	0.8850	0.0834
GaussianShader [16]	16.82	0.8912	0.0982	17.99	0.8969	0.0831	26.47	0.8826	0.0822
Ours	**22.71**	**0.9366**	**0.0498**	**23.15**	**0.9694**	**0.0373**	**30.84**	**0.9032**	**0.0763**
Ours w/o unified	21.95	0.8816	0.0554	21.92	0.8934	0.0495	26.95	0.8832	0.0824
Ours w/o constrain	22.07	0.9027	0.0513	22.82	0.9259	0.0408	28.12	0.8934	0.0924

under diverse lighting scenarios show consistent adjustments, preserving the material attributes while accurately reflecting the changes in illumination. We have demonstrated a superior relighting ability compared to other baselines.

4.2.2. Materials and geometry

Surface Normal. We present the surface normal comparison results in Figure 7. Our model yields the best performance in normal estimation when compared to Wonder3D [28], RGB2X [47], and Yi et al, [46].

Albedo. For albedo estimation, we have compared 2 optimization-based methods and 6 data-driven methods in terms of PSNR, SSIM and LPIPS. Figure 4 presents the qualitative comparison for albedo estimation, and the quantitative comparison can be found in Table 2. Our method significantly outperforms other methods.

Metallic and Roughness. Since existing data-driven methods do not include metallic and roughness estimation, we have compared the roughness and metallic estimation with two optimization-based inverse rendering method, including NVdiffRec [31] amd GaussianShader [16], the quantitative results can be found in Table 2. More visualization cases can be found in the supplementary material.

4.2.3. Real-World Inversing

Despite being trained on synthetic data, our model is capable of performing inverse rendering in real-world cases. As shown in Figure 8, the "Phone stand" case has a more metallic appearance while maintaining some roughness. It is under a white-to-yellow light source thereby showing a white-to-yellow color in lighting. Our model also excels at recovering high-frequency ambient lighting from the image as specular lighting, as demonstrated in the "kettle" case.

4.3. Ablation Study

Dual Stream Diffusion. To demonstrate the effectiveness of our dual stream diffusion framework, we compared results with two separate diffusion models for performing rendering and inverse rendering. Combining the two models in a sin-

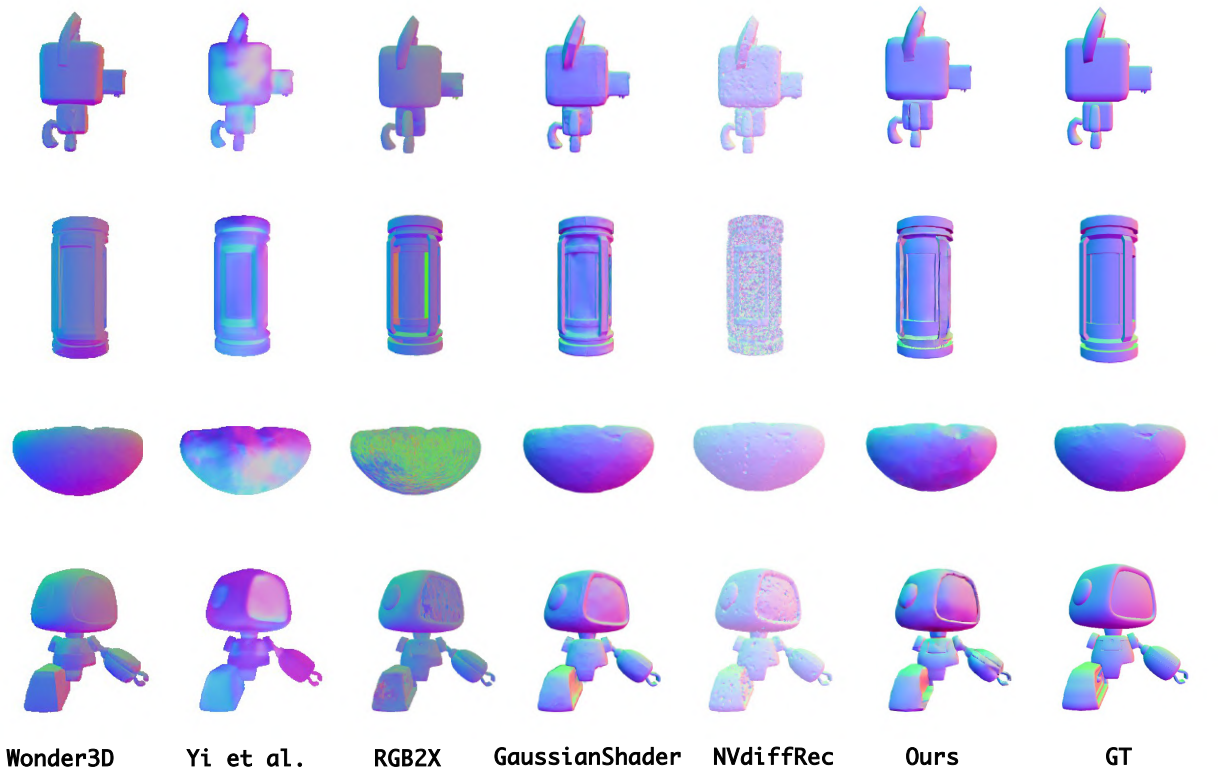

Figure 7. **Normal Comparison.** Normal Comparison of Uni-Renderer with others methods. **Best viewed in color.**

gle pipeline for both rendering and inverse can benefit each other, as shown in Table 2, 3 where the jointly trained model performance prevails. From a multi-task learning perspective, the unified pipeline benefits each other by sharing the knowledge through model weights.

Cycle-Consistent Constraint. We also conducted experiments to show the effectiveness of the cycle-consistent constraint. The constrain can mitigate the ambiguity problem by enforcing consistency between the intrinsic properties and the images, leading to improved performance. The results in Table 2, 3 demonstrated the inverse rendering quality improves with the constrain term being included.

5. Conclusion

We present a method that allows photo-realistic rendering and inverse rendering in a unified framework. Our method demonstrates strong performance in real-world cases, achieving photo-realistic rendering and inverse rendering that generalizes well beyond synthetic training data. Despite this success, there are still limitations when applying our approach to certain real-world scenarios, primarily due to the domain gap between synthetic training data and real-world images. This gap can lead to challenges in accurately handling complex or unfamiliar objects and environments, where the synthetic-to-real adaptation is less effective. To address this, future work will focus on incorporating more real-world data into our training process. By expanding our dataset with real-world examples, we aim to enhance the model's ability to bridge the synthetic-real domain gap, enabling better generalization and higher-quality rendering results across diverse real-world scenes.

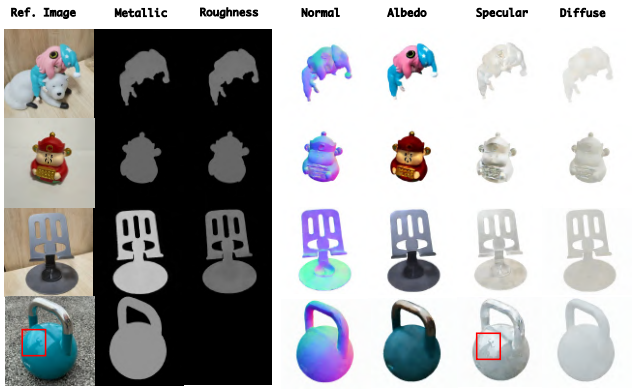

Figure 8. **Real World Inversing.** We present different real-world inversing cases under different lighting conditions. **Best viewed in color.**

References

[1] Fan Bao, Shen Nie, Kaiwen Xue, Chongxuan Li, Shi Pu, Yaole Wang, Gang Yue, Yue Cao, Hang Su, and Jun Zhu. One transformer fits all distributions in multi-modal diffusion at scale, 2023. 2, 3, 4, 5

[2] Jonathan T Barron and Jitendra Malik. Shape, illumination, and reflectance from shading. *IEEE transactions on pattern analysis and machine intelligence (TPAMI)*, 2014. 2

[3] Tim Brooks, Aleksander Holynski, and Alexei A. Efros. Instructpix2pix: Learning to follow image editing instructions, 2023. 6

[4] Chris Careaga and Yağız Aksoy. Intrinsic image decomposition via ordinal shading. *ACM Trans. Graph.*, 43(1), 2023. 2, 3, 7

[5] Wenzheng Chen, Joey Litalien, Jun Gao, Zian Wang, Clement Fuji Tsang, Sameh Khamis, Or Litany, and Sanja Fidler. Dib-r++: learning to predict lighting and material with a hybrid differentiable renderer. *Advances in Neural Information Processing Systems (NeuIPS)*, 2021. 2

[6] Matt Deitke, Dustin Schwenk, Jordi Salvador, Luca Weihs, Oscar Michel, Eli VanderBilt, Ludwig Schmidt, Kiana Ehsani, Aniruddha Kembhavi, and Ali Farhadi. Objaverse: A universe of annotated 3d objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 6

[7] Valentin Deschaintre, Miika Aittala, Fredo Durand, George Drettakis, and Adrien Bousseau. Single-image svbrdf capture with a rendering-aware deep network. *ACM Transactions on Graphics (ToG)*, 2018. 2, 3

[8] Cass Everitt and Mark J Kilgard. Practical and robust stenciled shadow volumes for hardware-accelerated rendering. 2003. 2

[9] Yue Fan, Ivan Skorokhodov, Oleg Voynov, Savva Ignatyev, Evgeny Burnaev, Peter Wonka, and Yiqun Wang. Factored-neus: Reconstructing surfaces, illumination, and materials of possibly glossy objects. *arXiv preprint arXiv:2305.17929*, 2023. 3

[10] Roger Grosse, Micah K Johnson, Edward H Adelson, and William T Freeman. Ground truth dataset and baseline evaluations for intrinsic image algorithms. In *Proceedings of the IEEE/CVF International Conference on Computer Vision(ICCV)*, 2009. 2

[11] Yu Guo, Cameron Smith, Miloš Hašan, Kalyan Sunkavalli, and Shuang Zhao. Materialgan: Reflectance capture using a generative svbrdf model. *arXiv preprint arXiv:2010.00114*, 2020. 2, 3

[12] John K Haas. A history of the unity game engine. *Diss. Worcester Polytechnic Institute*, 2014. 2

[13] Wolfgang Heidrich and Hans-Peter Seidel. Realistic, hardware-accelerated shading and lighting. In *Proceedings of the 26th annual conference on Computer graphics and interactive techniques*, 1999. 2

[14] Roland Hess. *Blender foundations: The essential guide to learning blender 2.5*. 2013. 2

[15] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020. 3

[16] Yingwenqi Jiang, Jiadong Tu, Yuan Liu, Xifeng Gao, Xiaoxiao Long, Wenping Wang, and Yuexin Ma. Gaussianshader: 3d gaussian splatting with shading functions for reflective surfaces. *arXiv preprint arXiv:2311.17977*, 2023. 6, 7

[17] Kaggle user: dimensi0n. Lhq-1024 dataset. https://www.kaggle.com/datasets/dimensi0n/lhq-1024, 2023. 6

[18] James T Kajiya. The rendering equation. In *Proceedings of the 13th annual conference on Computer graphics and interactive techniques*, 1986. 2, 4

[19] Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Analyzing and improving the image quality of stylegan. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2020. 2

[20] Hiroharu Kato, Yoshitaka Ushiku, and Tatsuya Harada. Neural 3d mesh renderer. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2018. 2

[21] Mark J Kilgard and Jeff Bolz. Gpu-accelerated path rendering. *ACM Transactions on Graphics (TOG)*, 2012. 2

[22] Peter Kocsis, Vincent Sitzmann, and Matthias Nießner. Intrinsic image diffusion for indoor single-view material estimation, 2024. 2, 3, 7

[23] Jaroslav Křivánek, Christophe Chevallier, Vladimir Koylazov, Ondřej Karlík, Henrik Wann Jensen, and Thomas Ludwig. Realistic rendering in architecture and product visualization. 2018. 2

[24] Eric P Lafortune and Yves D Willems. Bi-directional path tracing. 1993. 2

[25] Michael Lewis and Jeffrey Jacobson. Game engines. *Communications of the ACM*, 2002. 2

[26] Ruofan Liang, Huiting Chen, Chunlin Li, Fan Chen, Selvakumar Panneer, and Nandita Vijaykumar. Envidr: Implicit differentiable renderer with neural environment lighting. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. 3

[27] Yuan Liu, Peng Wang, Cheng Lin, Xiaoxiao Long, Jiepeng Wang, Lingjie Liu, Taku Komura, and Wenping Wang. Nero: Neural geometry and brdf reconstruction of reflective objects from multiview images. In *SIGGRAPH*, 2023. 3

[28] Xiaoxiao Long, Yuan-Chen Guo, Cheng Lin, Yuan Liu, Zhiyang Dou, Lingjie Liu, Yuexin Ma, Song-Hai Zhang, Marc Habermann, Christian Theobalt, and Wenping Wang. Wonder3d: Single image to 3d using cross-domain diffusion, 2023. 7

[29] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 2021. 2

[30] Ron Mokady, Amir Hertz, Kfir Aberman, Yael Pritch, and Daniel Cohen-Or. Null-text inversion for editing real images using guided diffusion models, 2022. 6

[31] Jacob Munkberg, Jon Hasselgren, Tianchang Shen, Jun Gao, Wenzheng Chen, Alex Evans, Thomas Müller, and Sanja Fidler. Extracting Triangular 3D Models, Materials, and Lighting From Images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8280–8290, 2022. 6, 7

[32] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning*, pages 8162–8171. PMLR, 2021. 3

[33] Merlin Nimier-David, Delio Vicini, Tizian Zeltner, and Wenzel Jakob. Mitsuba 2: A retargetable forward and inverse renderer. *ACM Transactions on Graphics (TOG)*, 2019. 2

[34] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically based rendering: From theory to implementation*. 2023. 2

[35] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022. 3

[36] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models, 2022. 5

[37] Prafull Sharma, Varun Jampani, Yuanzhen Li, Xuhui Jia, Dmitry Lagun, Fredo Durand, William T. Freeman, and Mark Matthews. Alchemist: Parametric control of material properties with diffusion models, 2023. 3

[38] Peter Shirley and R Keith Morley. *Realistic ray tracing*. AK Peters, Ltd., 2008. 2

[39] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020. 3

[40] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020. 3

[41] J. Daniel Subias and Manuel Lagunas. In-the-wild material appearance editing using perceptual attributes, 2023. 6

[42] Shimon Vainer, Mark Boss, Mathias Parger, Konstantin Kutsy, Dante De Nigris, Ciara Rowles, Nicolas Perony, and Simon Donné. Collaborative control for geometry-conditioned pbr image generation, 2024. 3

[43] Catherine Winder and Zahra Dowlatabadi. *Producing animation*. 2013. 2

[44] Chen Xi, Peng Sida, Yang Dongchen, Liu Yuan, Pan Bowen, Lv Chengfei, and Zhou. Xiaowei. Intrinsicanything: Learning diffusion priors for inverse rendering under unknown illumination. *arxiv: 2404.11593*, 2024. 2, 3, 7

[45] Yao Yao, Jingyang Zhang, Jingbo Liu, Yihang Qu, Tian Fang, David McKinnon, Yanghai Tsin, and Long Quan. Neilf: Neural incident light field for physically-based material estimation. In *European Conference on Computer Vision (ECCV)*, 2022. 2, 3

[46] Renjiao Yi, Chenyang Zhu, and Kai Xu. Weakly-supervised single-view image relighting, 2023. 2, 3, 7

[47] Zheng Zeng, Valentin Deschaintre, Iliyan Georgiev, Yannick Hold-Geoffroy, Yiwei Hu, Fujun Luan, Ling-Qi Yan, and Miloš Hašan. Rgb↔x: Image decomposition and synthesis using material- and lighting-aware diffusion models. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers '24*, page 1–11. ACM, 2024. 2, 3, 7

[48] Yuanqing Zhang, Jiaming Sun, Xingyi He, Huan Fu, Rongfei Jia, and Xiaowei Zhou. Modeling indirect illumination for inverse rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 3